

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sn

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn import metrics

np.random.seed(42)

n = 500

word_frequency = np.random.randint(1, 101, n)
message_length = np.random.randint(5, 501, n)

# Create spam score
score = (
    0.65 * word_frequency
    + 0.35 * (message_length / 5)
    + np.random.normal(0, 12, n)
)

# 1 = Spam, 0 = Not Spam
label = (score > 48).astype(int)

data = pd.DataFrame({
    'word_frequency': word_frequency,
    'message_length': message_length,
    'label': label
})

print("Dataset Shape:", data.shape)
print(data.head())

Dataset Shape: (500, 3)

```

	word_frequency	message_length	label
0	52	80	0
1	93	158	1
2	15	148	0
3	72	439	1
4	61	489	1

```

data.to_csv('spam_svm_500.csv', index=False)

print("CSV file created successfully!")

CSV file created successfully!

data.isnull().sum()

```

```

word_frequency    0
message_length    0
label             0
dtype: int64

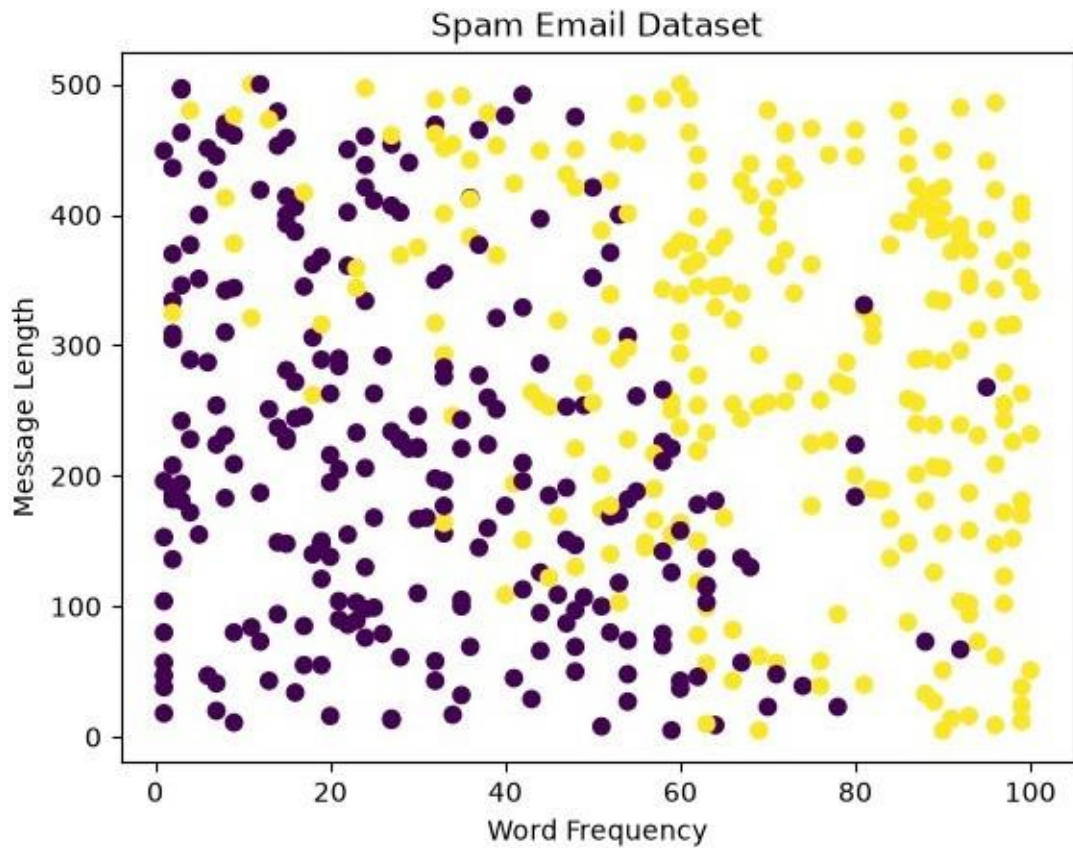
print(data['label'].value_counts())

label
1      256
0      244
Name: count, dtype: int64

plt.scatter(
    data['word_frequency'],
    data['message_length'],
    c=data['label']
)

plt.xlabel('Word Frequency')
plt.ylabel('Message Length')
plt.title('Spam Email Dataset')
plt.show()

```



```

x = data[['word_frequency', 'message_length']]
y = data['label']

x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.3,
    random_state=5
)

print("Training Data:", x_train.shape)
print("Training Labels:", y_train.shape)
print("Testing Data:", x_test.shape)
print("Testing Labels:", y_test.shape)

Training Data: (350, 2)
Training Labels: (350,)
Testing Data: (150, 2)
Testing Labels: (150,)

sc = StandardScaler()

x_train = sc.fit_transform(x_train)
x_test = sc.transform(x_test)

model = SVC(
    kernel='rbf',
    random_state=0
)

model.fit(x_train, y_train)

SVC(random_state=0)

svc_prediction = model.predict(x_test)

print("SVC Prediction:")
print(svc_prediction)

SVC Prediction:
[1 1 0 0 0 0 0 0 1 1 1 0 0 1 1 1 1 0 0 1 0 1 1 0 0 0 0 0 1 0 0 0 0 1 1
 1 0
 0 0 0 1 0 0 0 0 1 1 0 0 1 0 0 1 1 0 0 1 0 1 1 1 1 1 0 1 0 1 0 0 0 1 0
 1 0
 0 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 0 0 0 1 0 1 1 1 0 1 0 1 1 0
 0 1
 1 1 1 0 0 1 0 0 1 0 1 0 0 0 1 0 0 0 0 0 1 0 0 1 0 1 0 0 0 1 1 0 1 0 0
 1 0
 1 0]

```

```

conf_mat = metrics.confusion_matrix(
    y_test,
    svc_prediction
)

print("SVC [kernel - rbf]")
print("Confusion Matrix:")
print(conf_mat)

Accuracy_score = metrics.accuracy_score(
    y_test,
    svc_prediction
)

print("Accuracy Score:", Accuracy_score)

print(
    "Accuracy in Percentage:",
    int(Accuracy_score * 100),
    "%"
)

print("\nClassification Report:")
print(
    metrics.classification_report(
        y_test,
        svc_prediction
    )
)

SVC [kernel - rbf]
Confusion Matrix:
[[65  7]
 [17 61]]
Accuracy Score: 0.84
Accuracy in Percentage: 84 %

Classification Report:

```

	precision	recall	f1-score	support
0	0.79	0.90	0.84	72
1	0.90	0.78	0.84	78
accuracy			0.84	150
macro avg	0.84	0.84	0.84	150
weighted avg	0.85	0.84	0.84	150

```

conf_mat = pd.crosstab(
    y_test,
    svc_prediction,

```

```
    rownames=['Actual'],  
    colnames=['Predicted']  
)  
  
sn.heatmap(  
    conf_mat,  
    annot=True  
)  
  
plt.title('SVM - Spam Email Classification')  
plt.show()
```

